
UNA EIF400-II-2026 Paradigmas de Programación

Sesion: 01/09/2026

Autor: CLoria

Los ejercicios son por default para autoestudio no para entregar necesariamente

Ejercicios

1. Usando el modelo de combinadores de parsers desarrollado en la clase de hoy, haga los siguientes cominadores. Haga casos de prueba.

a. Un combinador `Star(parser)` que aplique el `parser` al input hasta que no sea posible. Por ejemplo, para un `source:InputString` como `" 123 abc 456 de f g 78 9 ]+10 "` el parser `Star( Or( id(), Number() ) )` devolvería `Ok(tokens, rest)` donde `tokens` tiene el array de los tokens asociados a `Token("123", "NUM")`, `Token("abc", "ID")`, hasta `Token("9", "NUM")`, y `rest` sería `InputString("+10 ", source.input)`. Donde `Or`, `Number`, `Id` son los combinadores hechos en clase. Si `parser` no cumple ya desde el inicio entonces devolvemos `Ok(source.input, source.index)`, es decir, como si no nada hubiera sido parseado. Note que es `eager`, trata de usar `parser` lo más que pueda.

b. Generalice a `Star(parser, max:number)` que a lo más parsea `max` tokens, es decir, nunca parsea más de esa cantidad de tokens. Si se omite el parámetro `max` esto equivale al comportamiento anterior (se obtiene todos).

c. Diseñe `While(parser, until)` que es como `Star` pero para cuando el source cumple con el filtro `until`. Nota: es como un `while` no como como un `do-while`.

2. Logre que `TS` acepte todo lo anterior.